

Algorithmic Problem Solving 2026

Project Report

Group	Group H
Members	Karl Thedor Ruby (krub@itu.dk) Kristoffer Mejborn Eliasson (krme@itu.dk) Lukas Shaghashvili-Johannessen (lush@itu.dk)
Supervisor	Holger Dell, Martin Aumüller
Course	Algorithmic Problem Solving (APS 2026)
Date	2026-05-22

Contents

1. Abstract	2
2. Designed Problem – The Siege	2
2.1. Problem description	2
2.2. Intended solution	3
2.3. Test-case design	4
2.4. Accepted and wrong solutions	5
2.5. Input validation & formatting	5
2.6. Verify notes	5
2.7. Testing and Validation	5
2.8. Source for Designed Problem	6
3. Solved Problems	6
3.1. Robert Hood (Geometric Algorithms)	6
3.1.1. Problem metadata	6
3.1.2. Solution overview	6
3.1.3. Correctness argument	7
3.1.4. Complexity and time-limit reasoning	7
3.1.5. Empirical worst-case testing	7
3.1.6. Implementation notes	7
3.2. Cookie Selection (Algorithms on Arrays)	8
3.2.1. Problem metadata	8
3.2.2. Solution overview	8
3.2.3. Correctness argument	8
3.2.4. Complexity and time-limit reasoning	8
3.2.5. Empirical worst-case testing	9
3.2.6. Implementation notes	9
3.3. Jagged Skyline (Randomized / Monte Carlo)	9
3.3.1. Problem metadata	9
3.3.2. Solution overview	9
3.3.3. Correctness argument	9
3.3.4. Complexity and time-limit reasoning	9
3.3.5. Empirical behavior and robustness	10
3.3.6. Implementation notes	10
3.3.7. References	10

1. Abstract

This report documents the algorithmic problem solving work of Group H for APS 2026.

We designed **The Siege**, a problem combining two classical techniques: maximum network flow and 0/1 knapsack. Each of N strongholds is modelled as a directed flow network; the gold yield equals the max-flow from source to sink. A warrior budget B constrains which strongholds can be conquered. The intended solution runs Edmonds-Karp per stronghold and solves the resulting knapsack with dynamic programming in $O(N \cdot VE^2 + NB)$ time. A brute-force $O(2^N)$ subset search and a greedy ratio heuristic serve as the time-limit-exceeded and wrong-answer foils respectively.

We solved three problems from the course problem set. **Robert Hood** (geometry) finds the diameter of a point set using Andrew's monotone chain convex hull followed by rotating calipers in $O(n \log n)$. **Cookie Selection** (algorithms on arrays) maintains a dynamic multiset under insertions and median queries using a Fenwick tree with coordinate compression and binary descent in $O(n \log n)$. **Free Real Estate** (randomized algorithms) applies a randomized approach to its core subproblem.

2. Designed Problem — The Siege

2.1. Problem description

You are Warchief Gruk, and today is a good day for war. Your Horde stands ready to crush the Dwarven Underkingdom, N strongholds carved deep into the mountains, each sitting on rich veins of gold.

Your goblin scouts (small, sneaky, and not terribly bright) have somehow wormed their way into every stronghold through cracks and drain pipes the dwarves forgot to seal. They came back clutching crumpled maps scratched onto hide scraps, giggling about “shiny rocks.” The maps are crude but surprisingly accurate: each one shows the layout of tunnels inside a stronghold.

Deep inside each stronghold sits the *Gold Vein* (room 0), where the richest ore waits to be mined. Once you conquer a stronghold, your goblin miners will dig out the gold and load it into mine carts that run through the tunnel network to the stronghold entrance (room $V_i - 1$), where your wolf-riders wait with wagons.

Each tunnel is *one-way*: mine carts can only travel in one direction through it, carved that way to keep traffic from jamming. Each tunnel also has a *weight limit*: the maximum kilograms of gold ore that a mine cart can carry through it per hour. Some tunnels are wide enough for a troll to stroll through; others are barely goblin-sized. Your miners will haul as much gold as physically possible from the vein to the entrance, using every tunnel to its limit simultaneously. You need to figure out how much gold each stronghold can yield.

But strongholds do not fall for free. The dwarves fight like cornered badgers behind their stone barricades. Each assault costs orc warriors their lives. You can muster exactly B warriors for the entire campaign, and you must conquer your chosen strongholds before the elven relief army arrives. Choose which strongholds to take to maximise the total gold your miners extract. Each stronghold can only be taken once.

Present your *Siege Plans* to the war council: the list of strongholds the Horde will conquer.

Input

The first line contains two integers N and B ($1 \leq N \leq 50$, $1 \leq B \leq 1000$), the number of dwarven strongholds and the number of orc warriors available.

Then N stronghold descriptions follow. Each stronghold starts with a line containing:

- a string s_i , the name of the stronghold (lowercase a–z, no whitespace, length ≤ 20),
- an integer c_i ($1 \leq c_i \leq B$), the number of orc warriors lost conquering it,
- an integer V_i ($2 \leq V_i \leq 15$), the number of rooms,
- an integer E_i ($1 \leq E_i \leq 40$), the number of tunnels.

The next E_i lines each describe one tunnel with three integers u, v, w ($0 \leq u, v < V_i, u \neq v, 1 \leq w \leq 100$), meaning a one-way tunnel from room u to room v with weight limit w .

Room 0 is the Gold Vein (source) and room $V_i - 1$ is the entrance (sink).

Output

Output the strongholds you should conquer to maximise total gold extracted without exceeding the warrior budget B . Print one line per selected stronghold in the format:

<index> <name>

where <index> is the 0-based position of the stronghold in the input. Lines must appear in ascending order of index. It is guaranteed that the optimal subset is unique.

Sample input 1:

```
3 10
ironkeep 3 3 2
0 1 5
1 2 2
stonewall 6 4 4
0 1 3
0 2 2
1 3 2
2 3 3
thornhold 4 3 2
0 1 4
1 2 3
```

Sample output 1:

```
1 stonewall
2 thornhold
```

Max-flows: ironkeep = 2 (bottleneck edge $1 \rightarrow 2$, cap 2), stonewall = 4 (two augmenting paths: $0 \rightarrow 1 \rightarrow 3$ flow 2, then $0 \rightarrow 2 \rightarrow 3$ flow 2), thornhold = 3. With $B = 10$: stonewall+thornhold costs $6 + 4 = 10 \leq 10$, gold $4 + 3 = 7$. ironkeep+stonewall costs 9, gold $2 + 4 = 6$. So optimal is stonewall+thornhold.

2.2. Intended solution

The problem decomposes into two independent subproblems applied sequentially:

1. **Per-stronghold max-flow.** For each stronghold i , compute the maximum flow from room 0 to room $V_i - 1$ using Edmonds-Karp (BFS-augmented Ford-Fulkerson). This gives the gold value g_i .
2. **0/1 Knapsack DP.** Treat each stronghold as a knapsack item with weight c_i and value g_i . Find the subset of items with total weight $\leq B$ that maximises total value.

```
function solve():
    for each stronghold i:
```

```

g[i] = edmonds_karp(graph_i, source=0, sink=V_i - 1)

dp[0..N][0..B] = 0          // dp[i][j] = best gold using first i
strongholds, budget j
for i in 1..N:
    for j in 0..B:
        dp[i][j] = dp[i-1][j]    // skip stronghold i
        if j >= c[i]:
            dp[i][j] = max(dp[i][j], g[i] + dp[i-1][j - c[i]])

// backtrack to recover selected indices
remaining = B
for i from N down to 1:
    if dp[i][remaining] != dp[i-1][remaining]:
        select stronghold i-1
        remaining -= c[i-1]
output selected in ascending order

```

Time complexity:

- Edmonds-Karp per stronghold: $O(V_i \cdot E_i^2)$. With $V_i \leq 15$, $E_i \leq 40$: at most $15 \times 40^2 = 24000$ operations each.
- All N max-flows: $O(N \cdot V_{\max} \cdot E_{\max}^2) \approx 50 \times 24000 = 1.2 \times 10^6$.
- Knapsack DP: $O(N \cdot B) = 50 \times 1000 = 50000$.
- **Overall:** $O(N \cdot VE^2 + NB)$ – effectively constant given the small bounds.

Memory: $O(V_{\max}^2)$ for the adjacency matrix per flow computation, $O(N \cdot B)$ for the DP table.

2.3. Test-case design

Sample tests (manually crafted, small enough for hand-verification):

- **1-small-manual:** 3 strongholds, $B = 10$. Tests basic knapsack selection; greedy by ratio picks the wrong pair.
- **2-medium-manual:** 3 strongholds with parallel-path graphs. Verifies that multi-path flow is summed correctly.
- **3-single-manual:** $N = 1$, tests single-stronghold boundary; the one stronghold is within budget so it is selected.
- **4-zeroflow-manual:** One stronghold has no s - t path (disconnected graph, flow = 0). Tests that a zero-gold stronghold is correctly skipped when a positive-gold alternative exists within budget.

Secret tests (generated by generators/generator.py or hand-crafted for specific corner cases):

- **1-single:** $N = 1$, B large – single item always selected if affordable.
- **2-small:** $N \leq 5$, $B \leq 20$ – brute-force can verify correctness.
- **3-medium:** $N \approx 20$, $B \approx 200$ – exercises DP without hitting any limit.
- **4-large:** $N = 50$, $B = 1000$ – maximum constraint stress test; brute-force 2^{50} is impossible.
- **5-tiebreak:** Constructed so two subsets tie in gold before one is broken by careful cost arrangement; validator enforces uniqueness, so this test confirms no two optimal subsets exist.
- **6-zeroflow:** Multiple strongholds with disconnected graphs (no s - t path, gold = 0); tests that zero-value items do not inflate the knapsack answer.
- **7-cycle:** Graphs containing cycles and back-edges; confirms that residual graph reverse-edge handling in Edmonds-Karp is correct (a bug here would under-count flow).

Corner cases covered: $N = 1$; B exactly equal to one item cost; all items affordable; no item affordable; zero flow; graphs that are chains, parallel paths, or contain cycles.

2.4. Accepted and wrong solutions

Accepted — `submissions/accepted/solution.py` (Python 3) and `solution.rs` (Rust): Edmonds-Karp max-flow per stronghold, then 0/1 knapsack DP with backtracking. Both produce identical output. The Python solution runs comfortably within 2 seconds given the small problem bounds; the Rust solution is included as a fast reference.

TLE — `submissions/time_limit_exceeded/brute_force.py` / `brute_force.rs`: Enumerates all 2^N subsets, computes total cost and gold for each, and keeps the best feasible subset. Correct for small N but $2^{50} \approx 10^{15}$ iterations makes it infeasible for $N = 50$. Used to verify correctness of the DP solution on small inputs.

WA — `submissions/wrong_answer/greedy.py`: Sorts strongholds by the gold-per-warrior ratio $\frac{g_i}{c_i}$ descending and greedily adds items as long as they fit in the remaining budget. This is the optimal strategy for **fractional** knapsack but fails for 0/1 knapsack. Counter-example from sample 2 ($B = 10$): ravencrest has ratio $\frac{5}{4} = 1.25$, wolfgate $\frac{4}{4} = 1.0$, dragonspire $\frac{6}{6} = 1.0$. Greedy picks ravencrest ($c = 4, g = 5$) then wolfgate ($c = 4, g = 4$), total gold = 9. Optimal is dragonspire + ravencrest ($c = 10, g = 11$): the lower-ratio dragonspire contributes more absolute gold than wolfgate, which the ratio heuristic cannot see.

2.5. Input validation & formatting

`input_validators/validator.py` enforces all constraints via regex and range checks, then runs a full knapsack over all test inputs to confirm the optimal solution is **unique** (exits with code 43 if ≥ 2 optimal subsets exist, exits with code 42 on success). Specific checks:

- Line 1 matches `^[1-9][0-9]* [1-9][0-9]*\n$`; $N \in [1, 50]$, $B \in [1, 1000]$.
- Each castle header: name lowercase letters only, length ≤ 20 , globally unique; $c_i \in [1, B]$; $V_i \in [2, 15]$; $E_i \in [1, 40]$.
- Each edge line: $u, v \in [0, V_i - 1]$, $u \neq v$, no duplicate (u, v) pair within the same stronghold; $w \in [1, 100]$.
- No trailing content after the last edge.

Contestants may assume all input is well-formed and within these bounds.

2.6. Verify notes

`verifyproblem` was run from the `problem/thesiege/` directory. All sample and secret test cases pass the validator (exit 42). The accepted solutions pass all tests. The brute-force solution is correctly flagged TLE on 4-large and the greedy is correctly flagged WA on 1-small-manual. No warnings were produced.

2.7. Testing and Validation

Run the accepted Python solution on a sample test:

```
python3 submissions/accepted/solution.py < data/sample/1-small-manual.in
```

Run the validator on an input (exit code 42 = valid and unique-optimal, 43 = invalid):

```
python3 input_validators/validator.py < data/sample/1-small-manual.in; echo "exit $?"
```

Generate a large random test and time the accepted solution:

```
python3 generators/generator.py 42 50 1000 15 40 100 > /tmp/large.in
time python3 submissions/accepted/solution.py < /tmp/large.in
```

Diff DP against brute-force on a small instance:

```
python3 generators/generator.py 7 5 20 8 15 50 > /tmp/small.in
python3 submissions/accepted/solution.py < /tmp/small.in > /tmp/dp.out
python3 submissions/time_limit_exceeded/brute_force.py < /tmp/small.in > /tmp/bf.out
diff /tmp/dp.out /tmp/bf.out
```

2.8. Source for Designed Problem

Repository layout under problem/thesiege/:

```
problem/thesiege/
  problem.yaml                                problem metadata (name, type, uuid)
  problem_statement/problem.en.tex            LaTeX problem statement
  submissions/
    accepted/solution.py                      Python 3 accepted solution
    accepted/solution.rs                      Rust accepted solution
    time_limit_exceeded/brute_force.py
    time_limit_exceeded/brute_force.rs
    wrong_answer/greedy.py
  generators/generator.py                     parametric random generator
  input_validators/validator.py               constraint + uniqueness checker
  data/
    sample/ 4 hand-crafted sample cases (.in / .ans)
    secret/ 7 generated/hand-crafted secret cases (.in / .ans)
```

Run locally with `verifyproblem` (requires the `problemtools` package):

```
cd problem/
verifyproblem thesiege/
```

3. Solved Problems

3.1. Robert Hood (Geometric Algorithms)

3.1.1. Problem metadata

- **Problem name / Kattis link:** [Robert Hood](#)
- **Short formal I/O description:** Given n points in the plane with integer coordinates, output the maximum Euclidean distance between any two points, as a floating-point number.

3.1.2. Solution overview

The maximum distance between any two points in a finite set is always achieved by two points on the convex hull of the set. This reduces the problem to two steps: compute the convex hull, then find the diameter of the hull polygon.

The convex hull is computed using Andrew's monotone chain algorithm, which sorts the points lexicographically and constructs the lower and upper hull in two linear passes. The result is the vertices of the hull in counter-clockwise order.

The diameter of the convex polygon is then found using the rotating calipers technique. For each edge of the hull, the algorithm advances an antipodal pointer j as long as the next vertex is farther from the edge than the current one. The cross product of the edge direction and the vector between consecutive

candidate points determines the direction to move j . Both endpoints of the current edge are checked against j at each step, so no antipodal pair is missed.

This approach was chosen because it achieves optimal asymptotic complexity and avoids the $O(m^2)$ brute-force over hull vertices, which could be slow when all input points lie on the hull.

3.1.3. Correctness argument

Reduction to hull. Any point strictly inside the convex hull cannot be a diameter endpoint, since projecting it outward to the hull boundary only increases distance. Therefore the maximum distance is realised by two hull vertices.

Rotating calipers. The algorithm maintains the invariant that j is the index of the vertex farthest from the directed edge between $\text{hull}[i]$ and $\text{hull}[i+1]$, among all indices not yet “passed” by i . The cross product check $cx > 0$ advances j when the next vertex is strictly farther, and stops otherwise. Because the hull is convex, the distance from an edge is unimodal over consecutive vertices, so the pointer j never needs to retreat. Both $\text{hull}[i]$ and $\text{hull}[(i+1) \% m]$ are compared to $\text{hull}[j]$ at each step to cover all antipodal pairs correctly.

Edge cases. When $m = 1$ (all points identical) the distance is 0. When $m = 2$ (collinear or just two distinct points) the distance is computed directly. These are handled before the calipers loop.

3.1.4. Complexity and time-limit reasoning

Let n be the number of input points and m the number of hull vertices, with $m \leq n$.

- **Convex hull:** $O(n \log n)$ dominated by the initial sort.
- **Rotating calipers:** $O(m)$ since each pointer advances at most m times.
- **Overall:** $O(n \log n)$ time and $O(n)$ space.

For $n = 10^5$, this is approximately $10^5 \times 17 \approx 1.7 \times 10^6$ operations for the sort, plus a linear pass. The implementation avoids square roots until the final step by comparing squared distances, eliminating floating-point overhead from the inner loop.

3.1.5. Empirical worst-case testing

The worst case for the rotating calipers occurs when all input points lie on the convex hull, maximising m . A set of n points uniformly distributed on a large circle achieves this. The following generator was used:

```
import math
N = 100000
print(N)
for i in range(N):
    angle = 2 * math.pi * i / N
    x = round(1e9 * math.cos(angle))
    y = round(1e9 * math.sin(angle))
    print(x, y)
```

Running this input on the solution completed in approximately 0.73 seconds in CPython 3, which is within the Kattis time limit.

3.1.6. Implementation notes

- **Language used:** Python 3
- **Files:** solutions/geometry/roberthood.py
- **How to run:** `python3 solutions/geometry/roberthood.py < input.txt`

3.2. Cookie Selection (Algorithms on Arrays)

3.2.1. Problem metadata

- **Problem name / Kattis link:** [Cookie Selection](#)
- **Short formal I/O description:** Given a sequence of up to 2×10^5 events, each either inserting a positive integer into a multiset or querying and removing the element at rank $\lfloor \frac{n}{2} \rfloor + 1$ (1-indexed) from the sorted multiset of current size n , output each queried value on its own line.

3.2.2. Solution overview

The core challenge is maintaining a dynamic multiset that supports two operations in sublinear time: insertion of an element and extraction of the element at a given rank. A naive sorted list achieves $O(n)$ per operation, which is too slow for $n = 2 \times 10^5$.

The solution uses a Fenwick tree over coordinate-compressed values. Because cookie diameters are up to 10^9 but there are at most 2×10^5 distinct values in the input, all values are mapped to contiguous indices $1, 2, \dots, m$ where m is the number of distinct values. The BIT stores counts: position i holds how many cookies with compressed value i are currently in the holding area. A prefix sum query then gives the number of cookies with value at most i .

To answer a rank query for rank k , the solution uses a binary descent on the BIT structure, finding the smallest index i such that the prefix sum up to i is at least k . This descent runs in $O(\log m)$ rather than $O(\log^2 m)$ compared to binary searching over prefix sum queries.

This approach was selected because the BIT offers simple, cache-friendly implementation while achieving the necessary $O(\log n)$ per operation.

3.2.3. Correctness argument

Coordinate compression. All values are read before processing begins. The mapping $\text{compress} : v \mapsto i + 1$ is a bijection from the set of distinct input values to $\{1, \dots, m\}$, preserving order. Thus rank in the compressed BIT equals rank in the original multiset.

BIT invariant. After each insertion of value v , the BIT count at position $\text{compress}(v)$ is incremented by one. After each removal, it is decremented by one. Therefore, at any point, $\text{bit.query}(i)$ equals the number of cookies currently in the holding area with compressed index at most i , which equals the number of cookies with diameter at most $\text{decompress}(i)$.

Rank formula. The problem specifies that for n cookies the cookie sent is at position $\lfloor \frac{n}{2} \rfloor + 1$ in sorted order for both odd and even n . Setting $\text{rank} = n // 2 + 1$ before each removal matches this specification exactly.

kth correctness. The binary descent in $\text{kth}(k)$ maintains the invariant that $\text{prefix_sum}[1..pos] < k$ throughout the loop and terminates with the smallest $pos + 1$ satisfying $\text{prefix_sum}[1..pos+1] \geq k$, which is exactly the k -th occupied position.

3.2.4. Complexity and time-limit reasoning

Let N be the number of input lines and $m \leq N$ the number of distinct cookie diameters.

- **Preprocessing:** sorting the distinct values takes $O(N \log N)$.
- **Per event:** each insertion and each rank query performs one BIT update or descent, each costing $O(\log m) = O(\log N)$.
- **Overall:** $O(N \log N)$ time and $O(N)$ space.

For $N = 2 \times 10^5$ and a typical time limit of 1-2 seconds, roughly $2 \times 10^5 \times 17 \approx 3.4 \times 10^6$ elementary operations are required, which is well within budget even in Python given the constant factor of the BIT operations.

3.2.5. Empirical worst-case testing

A worst-case input alternates insertions and removals to keep the holding area non-empty throughout. The following generator was used:

```
N = 100000
for i in range(1, N + 1):
    print(i)
for _ in range(N):
    print('#')
```

This produces 2×10^5 lines: 10^5 insertions followed by 10^5 queries, exercising both the coordinate compression and the BIT at maximum size. Running this input on the solution completed in approximately 0.67 seconds in CPython 3, which is within the Kattis time limit.

3.2.6. Implementation notes

- **Language used:** Python 3
- **Files:** solutions/arrays/cookieselection.py
- **How to run:** `python3 solutions/arrays/cookieselection.py < input.txt`

3.3. Jagged Skyline (Randomized / Monte Carlo)

3.3.1. Problem metadata

- **Problem name / Kattis link:** [Jagged Skyline](#)
- **Short formal I/O description:** Interactively discover the tallest building column in a skyline of width w and height h using height queries of the form “is there a building at column x of height at least y ?”. The interactor answers with a positive/negative response for each query.

3.3.2. Solution overview

- **Approach:** Monte Carlo sampling of columns combined with targeted binary searches. Randomly sample column order and probe increasingly higher heights only when a sample indicates promise; then binary-search that column’s height.
- **Intuition:** Randomizing the search order avoids worst-case adversarial layouts and often finds a near-optimal column quickly; once a tall column is found, further work is limited because heights are bounded by h .

3.3.3. Correctness argument

- **Soundness:** Each confirmed positive query (“building” at (x, y)) is verified by binary search to determine the exact height of column x . The algorithm maintains the current best-known column and height and only replaces it upon finding a taller column.
- **Randomized benefit:** Random shuffle of columns yields good expected-time behavior against arbitrary distributions of tall columns; while not deterministic, the approach reduces the probability of scanning many low columns first.

3.3.4. Complexity and time-limit reasoning

- **Queries per sampled column:** In the worst case, a sampled column incurs one upward probe plus $O(\log h)$ queries for the binary search when successful.

- **Overall:** Expected number of probes depends on how many columns must be sampled before finding the global maximum; for typical inputs the randomized order finds the tallest column quickly, bounding total queries by roughly $O(k \log h)$ where k is the number of promising columns encountered.

3.3.5. Empirical behavior and robustness

- **Practical performance:** The implementation uses an early-exit when current best height reaches h . Randomization prevents pathological ordering and makes runtime stable across many test cases.
- **Robustness:** Works well in interactive judges since it never makes assumptions about skyline structure and always verifies heights before reporting the answer.

3.3.6. Implementation notes

- **Language used:** Python 3
- **Files:** solutions/random/jaggedskyline.py
- **How to run:** Run against an interactive judge that speaks the protocol. Locally, pair the script with a test interactor or use judge-provided tools. The script prints queries of the form $? \ x \ y$ and expects a single-line response (e.g. building or other negative token), and finishes by printing $! \ x \ h$.

3.3.7. References

- **Source:** Monte Carlo sampling pattern implemented in solutions/random/jaggedskyline.py.